

Writing Data onto file using FileWriter

Introduction to FileWriter

FileWriter is a class in Java's `java.io` package that allows you to write character data (text) to files. It's one of several ways to write data to files in Java, specifically designed for writing text rather than binary data.

Key features of FileWriter:

- Writes text to files character by character
- Can append to existing files or overwrite them
- Works with characters, strings, and character arrays
- Must be closed after use to save the data properly

Writing Data Using FileWriter

There are multiple ways to write data to files in Java. Here, we'll focus on using **FileWriter** to write character data.

Example:

```
import java.io.*;

public class LaunchIO2 {
    public static void main(String[] args) throws IOException {
        // Path to our file
        String filePath = "D:\\telusko\\InputOutput\\java.txt";

        // Declare FileWriter outside try block so it's accessible in finally
        FileWriter writer = null;

        try {
            // Create a File object that references our file
            File file1 = new File(filePath);

            // Create a FileWriter for the file
            // The 'true' parameter means we'll append to the file if it exists
            // If 'false' or omitted, it will overwrite the file
            writer = new FileWriter(file1, true);

            // Write a string to the file
            writer.write("Java");

            // Write a newline character
            writer.write("\n");

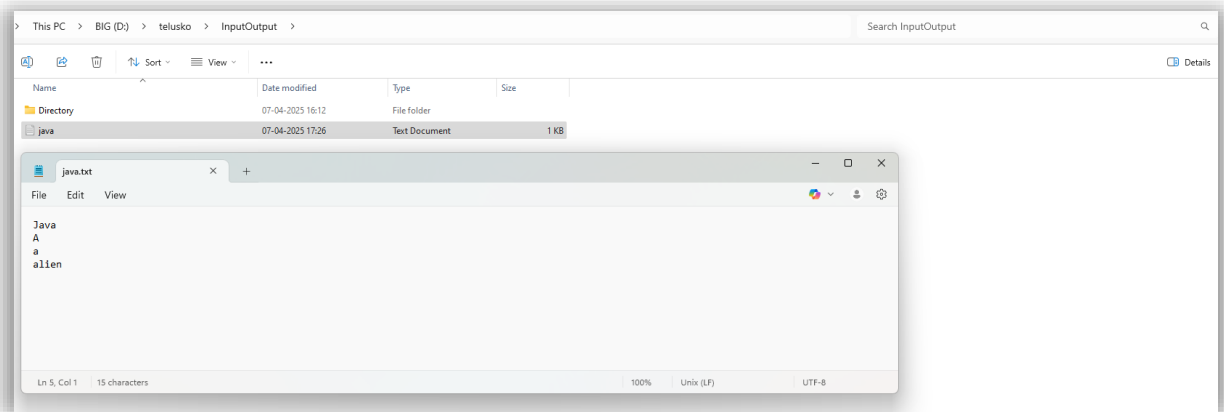
            // Write a character by its ASCII/Unicode value (65 = 'A')
            writer.write(65);
            writer.write("\n");

            // Write another character (97 = 'a')
            writer.write(97);
            writer.write("\n");

            // Write an array of characters
            char[] ch = {'a', 'l', 'i', 'e', 'n'};
            writer.write(ch);
            writer.write("\n");
        } catch (Exception e) {
            System.out.println("Some problem: " + e.getMessage());
        } finally {
            // Always close the writer to save the data and free resources
            // This executes whether the try block completes normally or throws an exception
            if (writer != null) {
                writer.close();
            }
        }
    }
}
```

Output:

- After running this program, if you open the file at **D:\telusko\InputOutput\java.txt**, you'll see the following content:



👉 Explanation:

- 1. Setting Up:** We define the file path and declare a **FileWriter** variable.
- 2. Creating File Reference:** We create a **File** object that points to our target file.
- 3. Creating FileWriter:**
 - We create a new **FileWriter** with the file and set **true** for append mode.
 - Append mode (**true**) means new data will be added to the end of the file without erasing existing content.
 - If we used **false** or omitted this parameter, the file would be overwritten.
- 4. Writing Different Types of Data:**
 - **String:** **writer.write("Java")** writes the text "Java" to the file.
 - **Newline:** **writer.write("\n")** adds a line break.
 - **Character by ASCII value:** **writer.write(65)** writes 'A' (ASCII 65) to the file.
 - **Character by ASCII value:** **writer.write(97)** writes 'a' (ASCII 97) to the file.
 - **Character array:** **writer.write(ch)** writes all characters in the array 'alien' to the file.
- 5. Closing the Writer:**
 - In the **finally** block, we ensure the writer is closed with **writer.close()**.
 - This is critical because data might not be fully written to the file until the writer is closed.
 - The **finally** block guarantees this happens, even if an exception occurs.